Laboratório de Concorrência · Aula 10

Gustavo Fumagalli Bandeira · RA 24011199-2 · Turma B
Gerado em 14/10/2026, 21:34:17 · tempo registrado no cronômetro: 0:00

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

Sincronização é quando o um precisa esperar o outro terminar algo, um exemplo é passar itens no caixa quando tem fila para ser sua vez tem que esperar o da frente terminar para dai ser sua vez. Competição é quando dois querem a mesma coisa ao mesmo tempo, exemplo duas pessoas querem usar o único banheiro disponível.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

esperado: 2000000 obtido: 1008044

1ª execução (n = 1.000.000)

obtido: 1008044

2ª execução (n = 1.000.000)

obtido: 1026811

3ª execução (n = 1.000.000)

obtido: 1021716

Saída do programa

```
 1 execução com 1000: esperado: 2000 obtido:   2000
 2 execução com 1000: esperado: 2000 obtido:   2000
```

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Os valores não batem com o esperado, apenas quando o valor de n é trocado para 1000. Não os valores não foram iguais nas três primeiras execuções, muda porque o escalonador decide quem roda e quando, e isso nunca é igual.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

A thread A lê o contador = 10 , antes dela gravar, a B também lê 10. A soma e grava 11, B soma e grava 11 também. Duas somas foram feitas, mas o contador só subiu 1, então uma se perdeu.

1.3 Com n = 1000, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Com n = 1000 deu certo 2000, porque é tão rápido que as threads quase não se cruzam. O perigo é esse: nos testes pequenos tudo passa, mas em produção, com muito acesso ao mesmo tempo, o erro aparece do nada e difícil de reproduzir.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

AAABBB → 8

ABABAB → 6

AABBAB → 6

BBABAA → 4

AABABB → 6

ABAABB → 6

BBBAAA → 7

Saída do programa

```
Ordens possíveis: 20
TOTAL = 4:  9 ordens, por exemplo AABBBA
TOTAL = 6:  9 ordens, por exemplo AABABB
TOTAL = 7:  1 ordens, por exemplo BBBAAA
TOTAL = 8:  1 ordens, por exemplo AAABBB
```

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

São possíveis 20 ordens, 4 resultados diferentes sendo 4,6,7,8. Só uma ordem da o resultado 8 sendo AAABBB.

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Na ordem AABBBA, primeiro A lê 3 e calcula 4, B lê 3, calcula 6 e grava 6, Por ultimo A grava 4 por cima, A opressão de B simplesmente sumiu, foi sobrescrita.

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Acho que a sem sincronização vai ser mais rápida, mas vai dar o valor errado. Já as outras vão demorar mais, porem vão dar o valor certo.

Saída do programa

```
1 resultado:
esperado:           2000000
sem sincronizacao: 1008480 em 43 ms
synchronized:      2000000 em 131 ms
AtomicInteger:     2000000 em 76 ms

2 resultado:
esperado:           2000000
sem sincronizacao: 1023743 em 2 ms
synchronized:      2000000 em 116 ms
AtomicInteger:     2000000 em 74 ms
```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

Monitor e Operação atômica dão o mesmo valor de 2000000 e o Semáforo deu errado. Os corretos são mais lentos porque as threads precisam esperar a vez, e esse controle tem custo> A versão sem cuidado é rápida porque ninguém espera ninguém.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não resolve. Cada saldo fica certinho sozinho, mas a transferência são duas operações, tirar de uma e pôr na outra, e alguém pode ver ou mexer no meio do caminho. Precisa travar as duas contas juntas, tipo um synchronized ou lock envolvendo a operação inteira.

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

Acho que não, pois A segura x e espera o Y, o B segura o Y e espera o X, então as usa devem ficar travadas esperando uma pela outra.

Saídas (antes e depois da correção)

```
1 resultado:
IMPASSE: 2 threads presas
2 resultado:
A terminou
B terminou
sem impasse
```

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

A pegou o X e ficou esperando o Y. B pegou o Y e ficou esperando o X. Cada uma segura o que a outra precisa, então ninguém anda nunca.

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Se todo mundo pega primeiro X e depois Y, quem pegar o X primeiro consegue pegar o Y também, a outra só espera e depois roda. Não tem como formar o "círculo" de espera. Exemplo real: transferência bancária entre duas contas ao mesmo tempo em sentidos opostos (A->B e B->A).

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

Com 3 trabalhadores, uns 200ms com 2 rodadas de 100ms.Com 1, uns 600ms, porque faz tudo em fila.Com6, uns 100ms, já que cada um pega uma tarefa e roda tudo junto

Tempo com 1 trabalhador

600 ms

Tempo com 3 trabalhadores

200 ms

Tempo com 6 trabalhadores

100 ms

Saída do programa

```
trabalhador 1 fez a tarefa 2
trabalhador 2 fez a tarefa 3
trabalhador 3 fez a tarefa 1
trabalhador 3 fez a tarefa 6
trabalhador 2 fez a tarefa 5
trabalhador 1 fez a tarefa 4
3 trabalhadores, 6 tarefas de 100 ms: levou 200ms
```

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Com 1 trabalhador deu uns 600 ms (6 tarefas uma atrás da outra), com 3 uns 200 ms (2 rodadas) e com 6 uns 100 ms (tudo junto). Não precisa de trava porque ninguém mexe na mesma variável: as tarefas chegam por mensagem no canal, cada uma vai pra um trabalhador só.

Saídas (antes e depois da troca)

```
1 resultado: fatal error: all goroutines are asleep - deadlock!

2 resultado: 1
```

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

Num canal sem espaço, quem envia fica parado esperando alguém receber. Como o próprio main que envia é quem ia receber depois, ele trava esperando ele mesmo, e o Go acusa "all goroutines are asleep - deadlock!". Com espaço pra 1, a mensagem fica guardada no canal, o envio não trava e depois o main lê de boa.

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

Acho que o Rust nem vai deixar compilar, porque ele é chato com duas threads mexendo na mesma variável sem proteção.

Saídas (o erro e o resultado com move)

```
1 resultado: error[E0373]: closure may outlive the current function, but it borrows
`contador`, which is owned by the current function
 ⟶ HelloWorld.rs:7:27
  |
7 |     let t = thread::spawn(|| {
  |                           ^^ may outlive borrowed value `contador`

2 resultado 0
```

Saída do programa

```
Resultado contador = 400000
```

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

O move faz a thread levar uma cópia do número. Ela soma 1 na cópia dela, e o contador original do main continua 0.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

No Java o erro só apareceu rodando (e às vezes nem aparece). No Rust o compilador barrou antes de rodar. Vantagem: esse tipo de bug nem chega em produção. Custo: dá mais trabalho escrever, tem que usar Arc, Mutex e brigar com o compilador.

Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

Acho que sai A,D,C,B,A e D são códigos normal e rodam na hora; C é promessa e roda logo depois; B é setTimeout e vai pro fim da fila, mesmo com 0ms.

Saída do programa

```
// Prática 7 - JavaScript: uma única thread e um laço de eventos
// Cole em https://onecompiler.com/nodejs
console.log("A");
setTimeout(() ⇒ console.log("B"), 0);
Promise.resolve().then(() ⇒ console.log("C"));
console.log("D");

function ocupado(ms) {                        // laço que NÃO devolve o controle
  const fim = Date.now() + ms;
  while (Date.now() < fim) { }
}

setTimeout(() ⇒ {
  const t0 = Date.now();
  setTimeout(() ⇒ console.log("E: timer de 10 ms rodou depois de " + (Date.now() - t0) + "
ms"), 10);
  ocupado(300);                               // trava a única thread por 300 ms
}, 0);
```

7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

Saiu A, D, C, B. A e D são código normal, rodam na hora. C é promessa, que tem prioridade logo depois do código atual. B é setTimeout, que vai pra fila de timers e só roda depois, mesmo com 0 ms.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

Porque a única thread ficou presa 300 ms no laço ocupado, e o timer só roda quando ela fica livre. Num servidor Node.js, um cálculo pesado trava todo mundo: nenhum outro usuário é atendido enquanto ele não termina.

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

Condição de corrida: os dois leem "1 lugar livre" e os dois compram, vendendo o mesmo lugar duas vezes. Dá pra evitar fazendo "checar e reservar" numa operação só, travada (lock ou transação no banco), pra só um conseguir.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

A variedade de resultados possíveis.